LEARN CSS

- CLICK TO JUMP TO THE PAGE -

1. Learn the syntax of CSS.....5
Ruleset.....5
Selector.....5
Declaration block.....5
Property.....5
Value.....5
2. Learn how to target html elements.....6
Simple selectors.....6
Universal selector.....6
Element selector.....6
Class selector.....6
Id selector.....6
Grouping selector.....7
Combinator selectors.....7
Descendant Selector.....7
Child selector.....7
Combine selector types.....7
3. Learn where to add css styles.....8
Inline style.....8
Internal style.....8
External style.....8
4. Learn measurements.....9
Px.....9
Rems.....9
Ems.....9
Vh.....9
Vw.....10
Percentages.....10
5. Learn the box model.....11
Box-sizing.....11
Content-box.....11
Border-box.....11
Content.....12
Width.....12
Height.....12
Max-width.....12
Min-width.....12

Max-height....13
Min-height....13
Padding-top....13
Padding-right....13
Padding-bottom....13
Padding-left....13
Padding....14
Margin-top....14
Margin-right....14
Margin-bottom....14
Margin-left....14
Margin....15
Border....15
6. Learn how to layout elements....16
Display....16
None....16
Block....16
Inline....16
Inline block....16
Flex....17
Inline-flex....17
Grid....17
7. Learn Flexbox....17
Flex-direction....17
Justify-content....18
Align-items....18
Flex-start....18
Center....18
Flex-end....19
Space-around....19
Space-evenly....19
Space-between....19
Flex-wrap....19
Nowrap....20
Wrap....20
Align-content....20
Wrap-reverse....20
Flex flow....20
Flex-basis....21
Flex-grow....21
Flex-shrink....21
Flex....21

8. Learn Grid 22
Grid-template-columns 22
Grid-template-rows 22
Repeat() 22
Grid-auto-flow 22
Grid-auto-flow: Row 23
Grid-auto-flow: Column 23
Grid-auto-flow: Dense 23
Grid-auto-flow: row/column dense 23
Grid-auto-columns 23
Grid-auto-rows 24
Grid-column-start 24
Grid-column-end 24
Grid-column 24
Grid-row-start 25
Grid-row-end 25
Grid-row 25
Grid-area 25
Grid-template-areas 26
column-gap 26
row-gap 26
gap 26
Justify-items 26
Align-items 27
Start 27
End 27
Center 27
Stretch 28
Baseline 28
Justify-self 28
Align-self 28
9. Learn how to position elements 29
Position 29
Static 29
Absolute 29
Relative 29
Fixed 30
Sticky 30
Z-index 30
10. Learn how to style text elements 31
Font-family 31
Importing external fonts 31

Font-size....31
Font-weight....32
Font-style....32
Color....32
Text-decoration-line....32
Text-decoration-color....32
Text-decoration-style....32
Text-decoration-thickness....33
Text-decoration....33
Text-align....33
Text-align-last....33
Vertical-align....33
Text-indent....33
Letter-spacing....34
Word-spacing....34
White-space....34
Line-height....34
Text-shadow....34
Text-transform....34
Text-overflow....35
11. Learn how to make responsive websites....36
Breakpoints....36
Media queries....36
Media queries: All....36
Media queries: Screen....37
Media queries: Print....37
Media queries: Speech....37
Media queries: Width....37
Media queries: Height....37
Media queries: Orientation....38
Media queries: Concatenation....38
Media queries: Orientation....38
Mobile first approach....38

1. Learn the syntax of CSS

Ruleset

It consists of a selector and a declaration block.

Back to index

```
p {
  color: red;
  background-color: white;
}
```

Selector

It is used to target the specific html element or elements that are to be styled. There are many kinds of selectors, to know more about them go to **Learn how to target html elements**.

Back to index

```
p {
}
```

Declaration block

It is an ordered collection of CSS properties and values. The beginning and end of a declaration is marked by curly brackets.

Back to index

```
p {
  color: red;
  background-color: white;
}
```

Property

It defines which style or behaviour wants to be given to the HTML element. The properties are always written lowercase, and compound word properties are separated by a dash (-).
Properties and their values must be separated by colons.

Back to index

```
p {
  color: red;
  background-color: white;
}
```

Value

It defines the specific styling or behaviour for a property.
There must always be a semicolon after the value, otherwise the changes will not be applied.

Back to index

```
p {
  color: red;
  background-color: white;
}
```

2. Learn how to target html elements

Simple selectors

Targeting elements by element, class, id or the universal selector.

Back to index

```
p, a, li {
  color: red;
  background-color: white;
}
```

Universal selector

Targets all the html elements. It has the lowest specificity (0, 0, 0).

Back to index

```
* {
  color: red;
  background-color: white;
}
```

Element selector

Targets the html elements by using the **html tag**. The ruleset will apply to all elements that share the same tag. Their specificity is (0, 0, 1).

Back to index

```
p {
  color: red;
  background-color: white;
}
```

Class selector

Targets the html elements by the value passed in the **class attribute**. A dot is needed to indicate that a class selector is used. Their specificity is (0, 1, 0).

Back to index

```
.hero {
  color: red;
  background-color: white;
}
```

Id selector

Targets the html elements by the value passed in the **id attribute**. A hash is needed to indicate that a class selector is used. Their specificity is (1, 0, 0).

Back to index

```
#hero {
  color: red;
  background-color: white;
}
```

Grouping selector

When several elements should share the same styles, the selectors can be grouped together. Simply separate each selector with a comma.

Back to index

```
p, .hero, #hero {
  color: red;
  background-color: white;
}
```

Combinator selectors

Targeting elements based on a specific relationship between each other. Combining selectors can increase the specificity, so using two element selectors will make the specificity be (0,0,2).

Back to index

```
ul > li {
  color: red;
  background-color: white;
}
```

Descendant Selector

By leaving a space between the selectors, the last one written targets all the html elements with the same tag, class or id that are a descendant of the selectors that precedes it.

Back to index

```
section li {
  color: red;
  background-color: white;
}
```

Child selector

By using **>** between the selectors, the last one written targets all the html elements with the same tag, class or id that are a direct child of the selectors that precedes it.

Back to index

```
ul > li {
  color: red;
  background-color: white;
}
```

Combine selector types

In the same way that two element selectors can be combined, so can be done between classes and ids.

Back to index

```
#section1 > .paragraphs a {
  color: red;
  background-color: white;
}
```

3. Learn where to add css styles

Inline style

The css is written inside the html file, in the opening tag of the element targeted by the style attribute. As it makes the code hard to maintain, it's an approach best avoided.

Back to index

```
<p style="color: red;" ></p>
```

Internal style

It is written inside the html file, inside the **<head>** element by using the **<style>** tag.
It is not considered a good practice.

Back to index

```
<head>
 <style>
   p {
     color: red;
     background-color: white;
   }
 </style>
</head>
```

External style

Uses an **external css file** linked to one or several html documents by adding the **<link> tag** in the head. In the **link element** the **rel attribute** marks the type of relationship, and the **href attribute** the relative path to the css file.

Back to index

index.html

```
<head>
 <link rel="stylesheet" href="styles.css" />
</head>
```

styles.css

```
p {
  color: red;
  background-color: white;
}
```

4. Learn measurements

Px

It is an absolute measure and the best practice is to only use it when precision matters, as in properties like: **border**, **border-radius** and **max** or **min widths** and **heights**.

Back to index

```
p {
  max-width: 400px;
  min-width: 100px;
  border: 1px solid black;
  border-radius: 8px;
}
```

Rems

They are relative to the font-size of the html element, so 1rem will be equal to the pixels the html font-size is set to be. If there is none specified, it will use the browsers default (usually 16px).
Rems are useful for sizing text elements and the elements around them: **margins**, **paddings**, **width**, **height,** etc.

Back to index

```
html {
  font-size: 20px;
}

p {
  margin: 2rem; -> 40px
  padding: 1rem; -> 20px
  font-size: 0.5rem; -> 10px
}
```

Ems

They are relative to their own **font-size**, so **1em** is equal to the amount of **pixels the font-size** of the **element itself is set to be**. If there's none in the element, it will inherit a declared font-size from the closest parent element until the html element is reached. If no element has a font-size it will use the browser's default
Ems can be used in the same cases as **Rems**.

Back to index

```
section {
  font-size: 16px;
}

section > ul {
  font-size: 10px;
  margin: 2em; -> 20px
}

section > img {
  margin: 1em; -> 16px
}
```

Vh

It represents a percentage of the height of the viewport. 1vh is 1% of the viewport height.

Back to index

```
.hero {
  width: 80%;
  margin: 5rem;
  height: 40vh;
}
```

Vw

It represents a percentage of the width of the viewport. 1vw is 1% of the viewport width.

Back to index

```
.hero {
  width: 80%;
  margin: 5rem;
  height: 40vh;
}
```

Percentages

It is relative to the width of the parent element, so the 100% of the element's width will be equal to the percentage, rems, ems or pixels the parent's width is set to be.
They are useful for sizing **widths, margins** and **paddings**. Percentage values often refer to measurements in the parent that are not obvious at first. Refer to documentation if unsure.

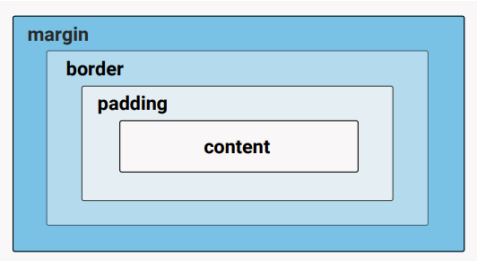
Back to index

```
section {
   width: 80%;
}

section > div {
  width: 100%;
}
```

5. Learn the box model

Every html element is wrapped by a box that primarily determines spacing and position of the element in the viewport. This box has four layers: content, padding, border and margin.
In the box model, to size elements accordingly the box-sizing property is fundamental.



Box-sizing

It is a property that alters how the total width and the total height of an element are calculated. The default value is content-box.

Back to index

```
p {
  box-sizing: content-box;
}
```

Content-box

The **width** and **height properties** only refer to the width and height of the **content**, so the **total width** and **height** will be calculated as:

total width = width + padding + border.
total height = height + padding + border

Back to index

```
p {
  border-sizing: content-box;
  width: 300px;
  height: 300px;
  padding: 30px;
  border: 1px solid red;
} -> the total width and height of the paragraph are 331px each.
```

Border-box

The **width** and **height properties** will include the content, the paddings and the border. So the **total width** and **height** will be calculated as:

total width = width
total height = height

Back to index

```
p {
  border-sizing: content-box;
  width: 300px;
  height: 300px;
  padding: 30px;
  border: 1px solid red;
} -> the total width and height of the paragraph are 300px each.
```

Content

It is where the text, images and other elements appear. The **width** and **height** properties are used to alter its size. **Be careful with the box-sizing property** when using **width and height!**

Back to index

```
p {
  width: 80%;
  margin: 2%;
  height: 20vh;
}
```

Width

Sets how much the width of an element is going to occupy. The **box-sizing** can affect how this property works.

Back to index

```
p {
  width: 80%;
}
```

Height

Sets how much the height of an element is going to occupy. The **box-sizing** can affect how this property works.

Back to index

```
p {
  height: 40vh;
}
```

Max-width

Defines the maximum width an element is going to occupy. The **box-sizing** can affect how this property works.

Back to index

```
p {
  max-width: 40rem;
}
```

Min-width

Defines the minimum width an element is going to occupy. The **box-sizing** can affect how this property works.

Back to index

```
p {
  min-width: 25m;
}
```

Max-height

Defines the minimum height an element is going to occupy. The **box-sizing** can affect how this property works.

Back to index

```
p {
  max-height: 40rem;
}
```

Min-height

Defines the maximum height an element is going to occupy. The **box-sizing** can affect how this property works.

Back to index

```
p {
  min-height: 40rem;
}
```

Padding-top

It is the space between the content and the top border.

Back to index

```
p {
  padding-top: 5%; -> applies to the top
}
```

Padding-right

It is the space between the content and the right border.

Back to index

```
p {
  padding-right: 5%; -> applies to the right
}
```

Padding-bottom

It is the space between the content and the bottom border.

Back to index

```
p {
  padding-bottom: 5%; -> applies to the bottom
}
```

Padding-left

It is the space between the content and the left border.

Back to index

```
p {
  padding-left: 5%; -> applies to the left
}
```

Padding

With the padding property, you have a shorthand syntax to describe the padding of an element, with the following patterns:

One value: padding <top+right+bottom+left>

Two values: `padding: <top+bottom> <left+right>`

Three values: `padding: <top> <left+right> <bottom>`

Four values: `padding: <top> <right> <bottom> <left>`

Back to index

```
p {
  padding: 5%; -> applies to all sides
  padding: 5% 3%; -> top-bottom and right-left.
  padding: 5% 2% 3%; -> top, right-left and bottom
  padding: 5% 2% 5% 3%; -> top, right, bottom, left
}
```

Margin-top

It is the space between the top border of the element and the other elements next to it.

Back to index

```
p {
  margin-top: 5%; -> applies to the top
}
```

Margin-right

It is the space between the right border of the element and the other elements next to it.

Back to index

```
p {
  margin-right: 5%; -> applies to the right
}
```

Margin-bottom

It is the space between the bottom border of the element and the other elements next to it.

Back to index

```
p {
  margin-bottom: 5%; -> applies to the bottom
}
```

Margin-left

It is the space between the left border of the element and the other elements next to it.

Back to index

```
p {
  margin-left: 5%; -> applies to the left
}
```

Margin

With the margin property, you have a shorthand syntax to describe the margin of an element, with the following patterns:

One value: margin <top+right+bottom+left>

Two values: margin: <top+bottom> <left+right>`

Three values: `margin: <top> <left+right> <bottom>`

Four values: `margin: <top> <right> <bottom> <left>`

Vertical margins can collapse with the margins of the previous and posterior elements.

Back to index

```
p {
  margin: 5%; -> applies to all sides
  margin: 5% 3%; -> top-bottom and right-left.
  margin: 5% 2% 3%; -> top, right-left and bottom
  margin: 5% 2% 5% 3%; -> top, right, bottom, left
}
```

Border

It is a border that surrounds the element. It is always there, even if it isn't visible.
Three values can be passed: the **measurement** (most often in pixels) for the size, the **type of decoration** (solid, dashed, dotted, none, etc) and the **colour**.

Back to index

```
p {
  border: 1px solid black;
}
```

6. Learn how to layout elements

Display

It specifies the display behaviour of an element, that is, how it is going to be laid out or if it is even going to be visible at all.

Back to index

```
p {
  display: block;
}
```

None

The element will not display.

Back to index

```
p {
  display: none;
}
```

Block

It is the default for most html elements. Block elements always start on a new line, take the whole width of the viewport, and are as high as the content inside.

Back to index

```
p {
  display: block;
}
```

Inline

It displays the element on the same line as the previous element, occupying only one line. Width and height will have no effect.
It is the default behaviour of some html elements, like the ****, **<a>**,**<b>**, and other text tags.

Back to index

```
li {
  display: inline;
}
```

Inline block

It combines behaviours of both displays. Like block elements, they can set their own width and height, and apply spacing to other elements with the top and bottom margins and paddings. And like inline elements, they stack horizontally in the same line.

Back to index

```
p {
 display: inline-block;
}
img {
 display: inline-block;
}
```

Flex

It turns the element into a flex-container, and all its direct children to flex-children.
Flex-containers behave like block elements, while flex-children position and size themselves according to the container they're in.
Flex contains a multitude of connected properties to alter the behaviour of flex-containers and flex-children. Go to **Learn flexbox** to learn which properties can modify the flex behaviour.

Back to index

```
div {
  display: flex;
}
```

Inline-flex

It turns the element into a flex-container, and all its direct children to flex-children.
Inline-flex containers behave like inline elements, while flex-children position and size themselves according to the container they're in.
Flex contains a multitude of connected properties to alter the behaviour of flex-containers and flex-children. There are different properties that can modify this layout behaviour.

Back to index

```
div {
  display: inline-flex;
}
div > p {
 width: 50%;
}
```

Grid

It displays the element as a block with a grid layout that defines how the children are going to layout within the element.

Back to index

```
p {
 display: grid;
}
```

7. Learn Flexbox

Flex-direction

It specifies how the element's children are going to be laid out within it, and determines the orientation of the **main axis**(horizontal in rows and vertical in columns) and the **cross axis**(vertical in rows and horizontal in columns) . The default value is row.

Back to index

```
div {
  display: flex;
  flex-direction: column-reverse;
}
```

Justify-content

It defines the alignment of the element's children along the **main axis**, which will be horizontal in rows and vertical in columns.

Back to index

```
div {
  display: flex;
  flex-direction: row;
  justify-content: center;
}
```

Align-items

It defines the alignment of the element's children along the **cross axis**, which will be vertical in the row and horizontal in the column.

Back to index

```
div {
  display: flex;
  flex-direction: row;
  align-items: flex-start;
}
```

Flex-start

It is the default value for both justify-content and align-items and positions elements at the start of the respective axis, See **justify-context** and **align-items** to know which axis they manipulate. Vertically aligns the items to the top, and horizontally to the left, with no space between them.

Back to index

```
div {
  display: flex;
  flex-direction: row;
  align-items: flex-start;
  justify-content: flex-start;
}
```

Center

It aligns the items to the center of the respective axis in **justify-context** and **align-items**.

Back to index

```
div {
  display: flex;
  align-items: center;
  justify-content: center;
}
```

Flex-end

It positions elements at the end of the respective axis in **justify-context** and **align-items**. Vertically aligns the items to the bottom, and horizontally to the right, with no space between them.

Back to index

```
div {
  display: flex;
  flex-direction: row;
  align-items: flex-end;
  justify-context: flex-end;
}
```

Space-around

Distributes spacing of the flex-container between all flex-children. Half the distance between children is allocated at flex-start and flex-end of the flex-container.

Back to index

```
div {
  display: flex;
  flex-direction: row;
  justify-context: space-around;
}
```

Space-evenly

Distributes the children with equal distance from each other and from the parent.

Back to index

```
div {
  display: flex;
  flex-direction: row;
  justify-context: space-evenly;
}
```

Space-between

Distributes spacing of the flex-container between all flex-children. No spacing is allocated to the flex-start or flex-end of the container.

Back to index

```
div {
  display: flex;
  flex-direction: row;
  justify-context: space-between;
}
```

Flex-wrap

It defines whether flex items can flow into a new line if the current line of the flex-container cannot fit all flex-children along the main axis.

Back to index

```
div {
  display: flex;
  flex-direction: row;
  flex-wrap: nowrap;
}
```

Nowrap

It's the default and forces the flex elements on a single line.

Back to index

```
div {
  display: flex;
  wrap: nowrap;
}
```

Wrap

It lets the children flow to a new line if there is not enough space for it, using the order in which they are written on the html.

Back to index

```
div {
  display: flex;
  wrap: wrap;
}
```

Align-content

It positions elements along the cross-axis like **align-items**, but takes effect when there are flex-children being wrapped.

Back to index

```
div {
  display: flex;
  flex-direction: row;
  align-content: center;
  wrap: wrap;
}
```

Wrap-reverse

It lets the children flow to a new line if there is not enough space for it but reverting the order in which they are declared in the html.

Back to index

```
div {
  display: flex;
  wrap: wrap-reverse;
}
```

Flex flow

It defines both the flex-direction and the flex-wrap properties respectively.

Back to index

```
div {
  display: flex;
  flex-flow: column wrap;
}
```

Flex-basis

It defines the initial width the flex children are going to occupy. Each child can have a different flex-basis, so the property is assigned to the children directly.

Back to index

```
div {
  display: flex;
  wrap: wrap;
  justify-content: space-around;
  align-items: center;
}
div > p {
 flex-basis: 100px;
}
```

Flex-grow

It defines the ability of a flex item (the children) to grow if necessary. The value is a number that will dictate the ratio of space available that the flex item is going to occupy.

Back to index

```
div {
  display: flex;
  wrap: column wrap;
}
div > p {
 flex-basis: 100px;
 flex-grow: 2;
}
```

Flex-shrink

It defines how much a flex item (the children) will shrink relative to the rest of flex items when there is no space on the row. The value is a number.

Back to index

```
div {
  display: flex;
  wrap: wrap;
}
div > p {
 flex-basis: 100px;
 flex-grow: 2;
 flex-shrink: 1;
}
```

Flex

It's the shorthand for the flex-grow, flex-shrink and flex basis respectively.

Back to index

```
div {
  display: flex;
  wrap: wrap;
}
div > p {
 flex: 2 1 100px;
}
```

8. Learn Grid

Grid-template-columns

Defines the columns with a space-separated list of values. The values represent the track size between the vertical grid lines. You can also name the vertical lines using bracket syntax. If the names are not added, the grid will auto name the column lines with positive numbers starting at 1.

Back to index

```
div {
  display: grid;
  grid-template-columns: 150px 1fr 10%;
  grid-template-columns: [first] 150px [line2] 1fr
  [line] 10%;
}
```

Grid-template-rows

Defines the rows with a space-separated list of values. The values represent the track size and the space of the grid line.You can also name the lines using bracket syntax. If the names are not added, the grid will auto name the column lines with positive numbers starting in 1.

Back to index

```
div {
  display: grid;
  grid-template-columns: 150px 200px 100px;
  grid-template-rows: repeat(3, 1fr);
}
```

Repeat()

This function used in the grid-template-columns and grid-template-rows will create a recurring pattern in the columns and rows. Inside the () you pass two values separated by a comma. The first is the number of repetitions this pattern will have and the second the size the columns or rows created will have.

Back to index

```
div {
  display: grid;
  grid-template-columns: 150px 200px 100px;
  grid-template-rows: repeat(3, 1fr);
}
```

Grid-auto-flow

Sets how the grid items (the children) are inserted in the grid. Row is the default.

Back to index

```
div {
  display: grid;
  grid-template-columns: 150px 200px 100px;
  grid-template-rows: repeat(3, 1fr);
  grid-auto-flow: column dense;
}
```

Grid-auto-flow: Row

It places items by filling the row first and jumping to a new row when there are no more cells left.

Back to index

```
div {
  display: grid;
  grid-template-columns: 150px 200px 100px;
  grid-template-rows: repeat(3, 1fr);
  grid-auto-flow: row;
}
```

Grid-auto-flow: Column

It places items by filling the column first and jumping to a new column when there are no more cells left.

Back to index

```
div {
  display: grid;
  grid-template-columns: 150px 200px 100px;
  grid-template-rows: repeat(3, 1fr);
  grid-auto-flow: column;
}
```

Grid-auto-flow: Dense

It places items by filling any holes in the grid.

Back to index

```
div {
  display: grid;
  grid-template-columns: 150px 200px 100px;
  grid-template-rows: repeat(3, 1fr);
  grid-auto-flow: dense;
}
```

Grid-auto-flow: row/column dense

The dense value can be added with a space after the row and the column values to specify in which direction to place the items while ensuring that all available spaces in the grid are filled, even if it means rearranging items to achieve a more compact layout.

Back to index

```
div {
  display: grid;
  grid-template-columns: 150px 200px 100px;
  grid-template-rows: repeat(3, 1fr);
  grid-auto-flow: column dense;
}
```

Grid-auto-columns

Sets a default width for all columns, but it will only affect those whose size is not set in the **template-grid-columns**.

Back to index

```
div {
  display: grid;
  grid-auto-flow: column;
  grid-template-rows: repeat(3, 1fr);
  grid-auto-columns: 1fr;
}
```

Grid-auto-rows

Sets a default width for all columns, but it will only affect those whose size is not set in the **template-grid-rows**.

Back to index

```
div {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 15rem;
}
```

Grid-column-start

Sets in which column the item will start or how many columns will occupy. You can pass the column line **name** for the first, or **span** plus the number of columns it will occupy.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
}
div > .p1 {
  grid-column-start: 2;
}
```

Grid-column-end

Sets in which column the item will end.You can pass the column line **name** to indicate in which line will end, or **span** plus the number of columns it will span.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
}
div >  .p1 {
  grid-column-start: 2;
  grid-column-end: span 2;
}
```

Grid-column

It is the shorthand command for the **grid-column-start** and **grid-column-end**

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
}
div >  .p1 {
  grid-column: 2 / span 2;
}
```

Grid-row-start

It works as the **grid-column-start** but for the row.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: 15rem 15rem 15rem;
}
div > .p1 {
  grid-row-start: 3;
}
```

Grid-row-end

It works as the **grid-column-end** but for the row.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: 15rem 15rem 15rem;
}
div > .p1 {
  grid-row-start: 1;
  grid-row-end: 3;
}
```

Grid-row

It is the shorthand command for the **grid-row-start** and **grid-row-end**

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
}
div > .p1 {
  grid-column: 1 / 3;
}
```

Grid-area

Sets the size and location of a specific grid item (child) in the grid. It can also assign a name to it. It is basically the shorthand for the **grid-row-start**, **grid-column-start**, **grid-row-end** and **grid-column-end** properties respectively.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
}
div > .p1 {
  grid-area: 1 / 2 / 3 / span 2;
}
```

Grid-template-areas

Defines a template by referencing the names of grid areas specified by the grid areas property. Repeating the name of a grid area causes it to span those cells, a period signifies an empty cell.

Back to index

```
div {
  display: grid;
  grid-template-areas: “header header header”;
  “main main . sidebar”;
}
div > .p1 {
  grid-area: header;
}
```

column-gap

Specifies the size between the grid column lines.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
  column-gap: 1rem;
}
```

row-gap

Specifies the size between the grid row lines.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
  row-gap: 1rem;
}
```

gap

Shorthand for the **column-gap** and **row-gap**. If no row-gap is specified, it is set to the same value as the column-gap.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
  gap: 1rem 1rem;
}
```

Justify-items

It defines the alignment of all the grid items along the inline axis (row). It can pass:
start - **end** - **center** - **stretch**

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
  justify-items: start;
}
```

Align-items

It defines the alignment of all the grid items along the block axis (column). It can pass:
start - **end** - **center** - **stretch** - **baseline**

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
  align-items: baseline;
}
```

Start

Aligns the items to be flush to the left in justify-items or justify-self and to the top in align-items or align-self.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
  align-items: start;
  justify-items: start;
}
```

End

Aligns the items to be flush to the right in justify-items, and to the bottom in align-items.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
  align-items: end;
  justify-items: end;
}
```

Center

Align the items to the center of the cell horizontally with justify-items and vertically with align-items.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
  align-items: center;
  justify-items: center;
}
```

Stretch

Fills the whole width of the cell in justify-items and the whole height of the cell in align-items.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
  align-items: stretch;
  justify-items: stretch;
}
```

Baseline

Aligns the items to the text baseline.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
  justify-items: baseline;
}
```

Justify-self

It works as **justify-items** but to align the item itself independently from the other grid items.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
}
div > .p1 {
  justify-self: start;
}
```

Align-self

It works as **align-items** but to align the item itself independently from the other grid items.

Back to index

```
div {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-auto-rows: 15rem;
}
div > .p1 {
  align-self: start;
}
```

9. Learn how to position elements

Position

It defines the positioning method used by an element. With position, to specify the exact position on the screen, we need to use the top, bottom, left and right properties. Not all of them have to be used for the position to work.

Back to index

```
nav {
  position: absolute ;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
}
```

Static

It is the default position for elements, following the normal flow of the page. It is not affected by the top, bottom, left and right properties.

Back to index

```
nav {
  position: static;
}
```

Absolute

It defines the position of the element relative to the nearest positioned ancestor or, if there are none, the body. It also removes the elements from the normal flow and can make them overlap other elements.

Back to index

```
nav {
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
}
```

Relative

It defines the position of the element relative to its normal position (the static). Other content will not be adjusted to fit in the gap left by the element.

Back to index

```
nav {
  position: relative;
  top: 0;
  left: 0;
}
```

Fixed

It defines the position of the element relative to the viewport, making it always stay in the same place. Fixed elements don't leave a gap, they can stack on top of or under other elements depending on their **z-index**.

Back to index

```
nav {
  position: fixed;
  top: 0;
  left: 0;
  z-index: 1;
}
```

Sticky

It defines the position of the element relative to the user's scroll position, making the element **toggle between relative and fixed position**: it will be relative until a given offset position is met in the viewport, and then it will be fixed.
Using top, bottom, left and/right will cause it to be adjusted out of its normal position.

Back to index

```
nav {
  position: sticky;
  top: 0;
  left: 0;
  z-index: 1;
}
```

Z-index

It defines the order of the positioned elements and its descendants along the z-axis. Elements with a higher z-index value will stack on top those with lower ones. Several values can be passed:

- **Auto** (default)
- **Number** -> positive and negative numbers are allowed. Elements with higher numbers are set on top of elements with lower ones.
- **Inherit** -> inherits the property of its parent element.

Back to index

```
nav {
  position: relative;
  top: 0;
  left: 0;
  z-index: 1;
}
main {
  position: relative;
  top: 0;
  left: 0;
  z-index: -1;
}
```

10. Learn how to style text elements

Font-family

Specifies the font of the text. It should hold several fonts as a fallback system, starting from the most specific to the most generic in case of browser incompatibility with one or several of them.
There are 5 generic font-families depending on the style: **serif**, **sans-serif**, **monospace**, **cursive** and **fantasy**.
Compound names should go in between quotation marks!

Back to index

```
html {
  font-family: "Times New Roman", Times, serif;
} -> fonts have small decorative strokes
html {
  font-family: Arial, Helvetica, Verdana, sans-serif;
} -> fonts have clean lines
html {
  font-family: "Courier New", Monaco, monospace;
} -> fonts have a mechanical look
html {
  font-family: "Lucida Handwriting", cursive;
} -> fonts imitate human handwriting
html {
  font-family: Copperplate, fantasy;
} -> fonts are decorative and playful
```

Importing external fonts

You can import fonts into your html from a source that provides you an external stylesheet that is accessible via an url.
You can also use the @font-face or the @import rule to import fonts via url or local files directly into your css stylesheet.

Back to index

index.html

```
<head>
  <link
      rel="stylesheet"
      href="https://fonts.googleapis.com/
      css?family=Sofia"
   />
</head>
```

styles.css

```
html {
  font-family: "Sofia", sans-serif;
}
```

Font-size

Specifies the size of the text.

Back to index

```
html {
  font-size: 16px;
}
```

Font-weight

Specifies the thickness of the font. You can use css predefined words like "bold", or "light", or you can use numbers, like 100, 500 or 800. The default value for regular font thickness is 400.

Back to index

```
html {
  font-weight: light ;
  font-weight: 100 ;
}
```

Font-style

Specifies if the font is in a specific style.

Back to index

```
html {
  font-style: normal ;
}
```

Color

Specifies the color of the text.

Back to index

```
html {
  color: black;
}
```

Text-decoration-line

Specifies if the text has a decoration line.

Back to index

```
html {
  text-decoration-line: line-through ;
}
```

Text-decoration-color

Specifies the color of the text's decoration line.

Back to index

```
html {
  text-decoration-color: purple;
}
```

Text-decoration-style

Specifies the style of the text's decoration line.

Back to index

```
html {
  text-decoration-style: dashed ;
}
```

Text-decoration-thickness

Specifies the thickness of the text's decoration line.

Back to index

```
html {
  text-decoration-thickness: auto;
  text-decoration-style: 3px;
}
```

Text-decoration

It's the shorthand of the **text-decoration-line**, **text-decoration-color**, **text-decoration-style** and **text-decoration-thickness** in that respective order. The first one is mandatory, but the other ones are optional.

Back to index

```
html {
  text-decoration: none;
  text-decoration: underline purple dashed 3px;
}
```

Text-align

Specifies the alignment of the text horizontally.

Back to index

```
p {
  text-align: right ;
}
```

Text-align-last

Specifies the alignment of the last line of the text horizontally.

Back to index

```
p {
  text-align-last: right ;
}
```

Vertical-align

Specifies the alignment of the last line of the text vertically.

Back to index

```
p {
  vertical-align: baseline ;
}
```

Text-indent

Specifies the indentation of the text.

Back to index

```
ul {
  text-indent: 50px;
}
```

Letter-spacing

Specifies the space between the characters in a text.

Back to index

```
p {
  letter-spacing: 5px;
  letter-spacing: -2px;
}
```

Word-spacing

Specifies the space between the words in a text.

Back to index

```
p {
  word-spacing: 10px;
  word-spacing: -2px;
}
```

White-space

Specifies how white space is handled in a text.

Back to index

```
p {
  white-space: wrap;
  white-space: nowrap;
}
```

Line-height

Specifies the space between the lines in a text.

Back to index

```
p {
  line-height: 0.8;
}
```

Text-shadow

Adds shadow to the text by passing the horizontal shadow and vertical shadow values respectively. The color of the shadow can be added as well, but it is optional.

Back to index

```
p {
  text-shadow: 2px 3px black;
}
```

Text-transform

Specifies the uppercase and lowercase letters in a text.

Back to index

```
p {
  text-transform: uppercase ;
}
```

Text-overflow

Specifies how overflowed content that is not displayed should be signalled to the user.

Back to index

```
p {
  text-overflow: string ;
}
```

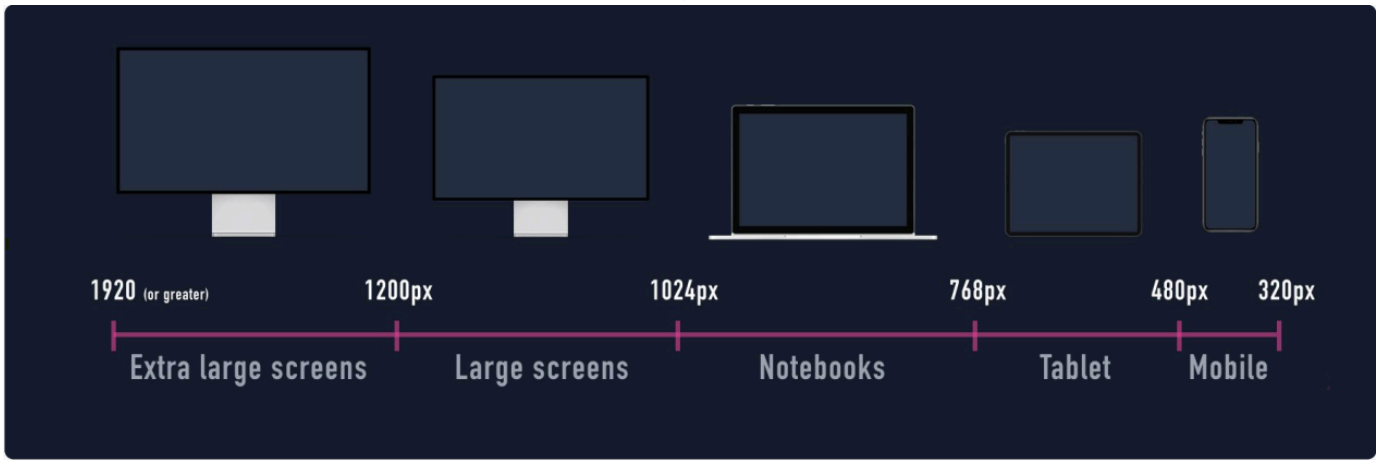
11. Learn how to make responsive websites

Breakpoints

They are thresholds where layout reorganisation might be needed due to limitations of relative units on screens that are either too small or too large.

Back to index

It's impossible to set breakpoints for every screen size, but the most common breakpoint ranges appear in the image below.



Media queries

Alter the styling of an element programmatically, making web pages truly responsive.
They consist of a media type and one or more expressions (conditions) that when they resolve as true override certain stylings.

Back to index

```
@media only screen and (max-width: 600px) {
  p {
    padding: 1rem;
  }
}
```

Media queries: All

Covers all media types.

Back to index

```
@media all {
  p {
    padding: 1rem;
  }
}
```

Media queries: Screen

Used for computers, tablets, smart-phones, etc.

Back to index

```
@media screen {
  p {
    padding: 1rem;
  }
}
```

Media queries: Print

Used for printers.

Back to index

```
@media print {
  p {
    padding: 1rem;
  }
}
```

Media queries: Speech

Used for screen readers that “read” the page out loud.

Back to index

```
@media speech {
  ul {
    speak-as: bullets;
  }
}
```

Media queries: Width

The **breakpoints** can be used to apply conditions about the width of the viewport. You can use the max-width and the min-width with the breakpoint to set the condition.

Back to index

```
@media (max-width: 600px){
  p {
    padding: 1rem;
  }
}
```

Media queries: Height

The **breakpoints** can be used to apply conditions about the height of the viewport. You can use the max-height and the min-height with the breakpoint to set the condition.

Back to index

```
@media (min-height: 900px){
  p {
    padding: 1rem;
  }
}
```

Media queries: Orientation

The orientation of the devices can also be used for the conditions, by passing the landscape or the portrait values.

Back to index

```
@media (orientation: landscape){
  p {
    padding: 1rem;
  }
}
```

Media queries: Concatenation

The media queries can use the connector and to concatenate conditions. The media will override the styling if they all apply.

Back to index

```
@media  screen and (min-width: 600px) and (max-width: 600px) and (orientation: landscape){
  p {
    padding: 1rem;
  }
}
```

Media queries: Orientation

The orientation of the devices can also be used for the conditions, by passing the landscape or the portrait values.

Back to index

```
@media  screen and (min-width: 600px) and (max-width: 600px) and (orientation: landscape){
  p {
    padding: 1rem;
  }
}
```

Mobile first approach

This is a designing concept where the design for the mobile goes before the design for desktop or any other device.

To translate it to media queries, the mobile css rulesets will not use a media query, but will be written first at the beginning of the css file. Once it is done, the media queries will go at the bottom of the css file, starting with the next smallest breakpoint (tablet) and continuing in this order till reaching the largest screen breakpoint.

Back to index

```
p {
  padding: 0.5rem;
}
@media  all and (min-width: 480px) {
  p {
    padding: 1rem;
  }
}
@media  all and (min-width: 768px) {
  p {
    padding: 1.5rem;
  }
}
```